BT5110 – Data Management & Warehousing

Group 39:

1. **James Sharon Susan**
2. **Keerthivasan Radhakrishnan**
3. **Louise**
4. **Sulistyo Agung Wibowo**

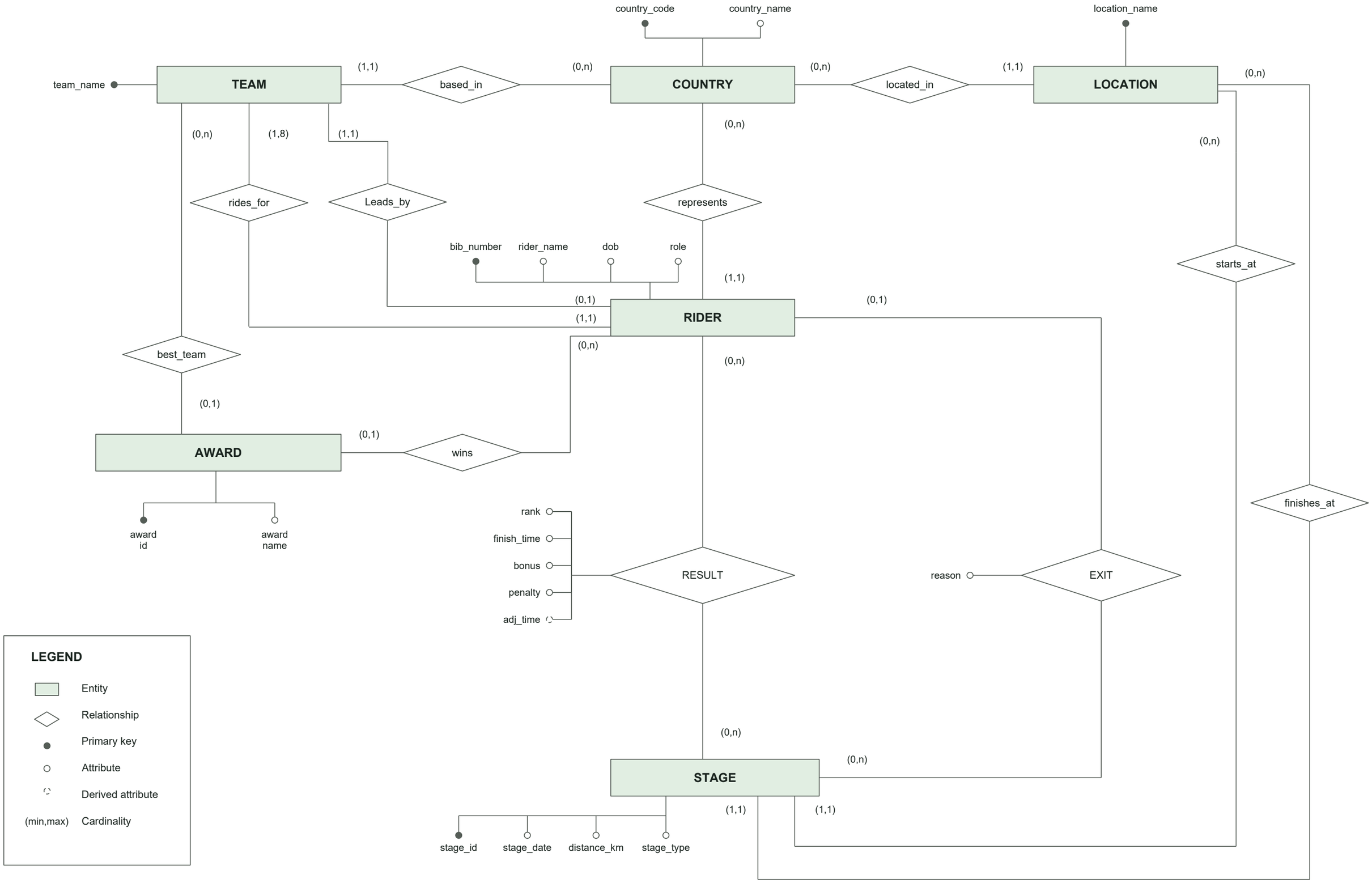
Tour de France Database and Analytics System

This group project develops a database and analytics system for the Tour de France, covering countries, teams, riders, stages, results, rider exits, and awards. It connects database design with practical analysis to organise race information and explore rider and team performance..

We used GPT-5.6 to describe specific uses: e.g., generate ideas, format paragraphs, improve expression, analyse effectiveness, create images and illustrations, produce drafts, refine, and/or finalise our assignment. We are responsible for the content and quality of the submitted work.



Project Task 1: Tour de France ER Diagram



The Constraints Not Capture in ERD:

- start_location ≠ finish_location for any stage
- If a rider exits at some stage S, no stage results should exist for that rider from stage S onwards

Project Task 2: Translate ER Diagram to Relational Schema

We used our Task 1 ER diagram as the starting point for the relational schema. Each entity became a table, and relationships were represented through foreign keys. Table 1 shows the resulting tables, their keys, and the rules that the data must follow.

Table 1. Relational Schema

Schema	R (attributes)	PK	FK	Constraints
COUNTRY	(country_code, country_name)	country_code	-	country_code is a 3-letter code. Countries may exist without teams or riders.
TEAM	(team_name, country_code)	team_name	country_code → COUNTRY	Each team belongs to one country and has 1–8 riders.
RIDER	(bib_number, rider_name, dob, role, is_leader, team_name, country_code)	bib_number	team_name → TEAM; country_code → COUNTRY	Each rider belongs to one team and represents one country. Exactly one rider per team has is_leader = TRUE.
LOCATION	(location_name, country_code)	location_name	country_code → COUNTRY	Each location belongs to one country and is used as a stage start or finish.
STAGE	(stage_id, stage_date, distance_km, stage_type, start_location, finish_location)	stage_id	start_location → LOCATION(location_name); finish_location → LOCATION(location_name)	Start and finish locations differ; distance_km > 0.
RESULT	(bib_number, stage_id, rank, finish_time, bonus, penalty)	(bib_number, stage_id)	bib_number → RIDER; stage_id → STAGE	At most one result per rider per stage. Rank is positive; time, bonus, and penalty are nonnegative. adj_time is calculated as finish_time − bonus + penalty.
EXIT	(bib_number, stage_id, reason)	bib_number	bib_number → RIDER; stage_id → STAGE	A rider exits at most once and has no results from the exit stage onward.
AWARD	(award_id, award_name, bib_number, team_name)	award_id	bib_number → RIDER; team_name → TEAM	Exactly one winner per award: either a rider or a team.

The many-to-many relationship between RIDER and STAGE is handled through the RESULT relation. Each result connects one rider to one stage with the foreign keys. The composite primary key, {bib_number, stage_id}, allows every rider to have results recorded for multiple stages and each stage to have results for multiple riders. This prevents duplicate results for the same rider-stage pair at the same time.

The following SQL code translates the relations in Table 1 into `CREATE TABLE` statements, including their primary keys, foreign keys, and constraints that can be defined directly in SQL.

```
CREATE TABLE country (
    country_code CHAR(3) PRIMARY KEY,
    country_name VARCHAR(100) NOT NULL,
    CHECK (country_code ~ '^[A-Z]{3}$')
);

CREATE TABLE team (
    team_name VARCHAR(100) PRIMARY KEY,
    country_code CHAR(3) NOT NULL,
    FOREIGN KEY (country_code) REFERENCES country(country_code)
);

CREATE TABLE rider (
    bib_number INTEGER PRIMARY KEY,
    rider_name VARCHAR(100) NOT NULL,
    dob DATE NOT NULL,
    role VARCHAR(50) NOT NULL,
    is_leader BOOLEAN NOT NULL,
    team_name VARCHAR(100) NOT NULL,
    country_code CHAR(3) NOT NULL,
    FOREIGN KEY (team_name) REFERENCES team(team_name),
    FOREIGN KEY (country_code) REFERENCES country(country_code)
);

CREATE TABLE location (
    location_name VARCHAR(100) PRIMARY KEY,
    country_code CHAR(3) NOT NULL,
    FOREIGN KEY (country_code) REFERENCES country(country_code)
);

CREATE TABLE stage (
    stage_id INTEGER PRIMARY KEY,
    stage_date DATE NOT NULL,
    distance_km NUMERIC(5, 2) NOT NULL,
    stage_type VARCHAR(30) NOT NULL,
    start_location VARCHAR(100) NOT NULL,
    finish_location VARCHAR(100) NOT NULL,
    FOREIGN KEY (start_location) REFERENCES location(location_name),
    FOREIGN KEY (finish_location) REFERENCES location(location_name),
    CHECK (distance_km > 0),
    CHECK (stage_type IN (
        'flat', 'hilly', 'mountain', 'individual time-trial'
    )),
    CHECK (start_location <> finish_location)
);
```

```
CREATE TABLE result (

    bib_number INTEGER NOT NULL,
    stage_id INTEGER NOT NULL,
    rank INTEGER NOT NULL,
    finish_time INTEGER NOT NULL,
    bonus INTEGER NOT NULL DEFAULT 0,
    penalty INTEGER NOT NULL DEFAULT 0,
    PRIMARY KEY (bib_number, stage_id),
    FOREIGN KEY (bib_number) REFERENCES rider(bib_number),
    FOREIGN KEY (stage_id) REFERENCES stage(stage_id),
    CHECK (rank > 0),
    CHECK (finish_time >= 0),
    CHECK (bonus >= 0),
    CHECK (penalty >= 0)
);

CREATE VIEW result_with_adj_time AS
SELECT
    bib_number,
    stage_id,
    rank,
    finish_time,
    bonus,
    penalty,
    (finish_time - bonus + penalty) AS adj_time
FROM result;

CREATE TABLE exit (
    bib_number INTEGER PRIMARY KEY,
    stage_id INTEGER NOT NULL,
    reason VARCHAR(255) NOT NULL,
    FOREIGN KEY (bib_number) REFERENCES rider(bib_number),
    FOREIGN KEY (stage_id) REFERENCES stage(stage_id)
);

CREATE TABLE award (
    award_id INTEGER PRIMARY KEY,
    award_name VARCHAR(100) NOT NULL,
    bib_number INTEGER,
    team_name VARCHAR(100),
    FOREIGN KEY (bib_number) REFERENCES rider(bib_number),
    FOREIGN KEY (team_name) REFERENCES team(team_name),
    CHECK (
        (bib_number IS NOT NULL AND team_name IS NULL)
        OR
        (bib_number IS NULL AND team_name IS NOT NULL)
    )
);
```

The SQL code implements the relations in Table 1 by defining their attributes, primary keys, foreign keys, and constraints that can be expressed directly in SQL.

The adjusted time (adj_time) is a derived attribute calculated as finish_time − bonus + penalty. Instead of storing it in the RESULT table, we create a SQL view named result_with_adj_time to calculate it dynamically. This avoids redundant data storage and ensures that adjusted time reflects any changes to finish_time, bonus, or penalty.

We then examine the functional dependencies in each table to check the normalization level. Table 2 shows these dependencies, together with the candidate keys and prime attributes.

Table 2. Functional Dependencies and Keys

Schema	Functional Dependency (FD)	Minimal Cover	Candidate key	Prime attribute(s)
COUNTRY	country_code → country_name	country_code → country_name	country_code	country_code
TEAM	team_name → country_code	team_name → country_code	team_name	team_name
RIDER	bib_number → rider_name, dob, role, is_leader, team_name, country_code	bib_number → rider_name; bib_number → dob; bib_number → role; bib_number → is_leader; bib_number → team_name; bib_number → country_code;	bib_number	bib_number
LOCATION	location_name → country_code	location_name → country_code	location_name	location_name
STAGE	stage_id → stage_date, distance_km, stage_type, start_location, finish_location	stage_id → stage_date; stage_id → distance_km; stage_id → stage_type; stage_id → start_location; stage_id → finish_location	stage_id	stage_id
RESULT	(bib_number, stage_id) → rank, finish_time, bonus, penalty	(bib_number, stage_id) → rank; (bib_number, stage_id) → finish_time; (bib_number, stage_id) → bonus; (bib_number, stage_id) → penalty	(bib_number, stage_id)	bib_number, stage_id
EXIT	bib_number → stage_id, reason	bib_number → stage_id; bib_number → reason	bib_number	bib_number
AWARD	award_id → award_name, bib_number, team_name	award_id → award_name; award_id → bib_number; award_id → team_name;	award_id	award_id

Functional Dependency Identification

We identified the functional dependencies based on the primary keys, relationships, and business rules defined in Table 1.

1. **COUNTRY:** country_code → country_name because each country code uniquely identifies one country and its name.

2. **TEAM:** team_name → country_code because each team has a unique name and belongs to exactly one country.

3. **RIDER:** bib_number → rider_name, dob, role, is_leader, team_name, country_code because each rider has a unique bib number that determines their personal information, team, and country they are representing.

4. **LOCATION:** location_name → country_code because each location has a unique name and belongs to one country.

5. **STAGE:** stage_id → stage_date, distance_km, stage_type, start_location, finish_location because each stage has a unique identifier that determines its date, distance, type, and start and finish locations.

6. **RESULT:** (bib_number, stage_id) → rank, finish_time, bonus, penalty because each rider can have at most one result per stage. Therefore, both attributes are required to uniquely determine the result.

7. **EXIT:** bib_number → stage_id, reason because each rider can exit the race at most once, so their bib number determines the exit stage and reason.

8. **AWARD:** award_id → award_name, bib_number, team_name because each award has a unique identifier that determines its name and the corresponding rider or team receiving it.

After identifying the functional dependencies, we derived the minimal cover by splitting dependencies with multiple right-hand-side attributes into individual dependencies. We then checked for redundant dependencies to ensure that each remaining FD was necessary.

Although bib_number → country_code in the RIDER relation and award_id → team_name in the AWARD relation may appear redundant when considering relationships across the entire database, we retain both dependencies in their respective minimal covers. Functional dependencies are evaluated within each relation based on its attributes and business rules. In RIDER, country_code represents the rider's nationality, which cannot necessarily be derived from team_name, as a rider may represent a different country from their team. Similarly, in AWARD, an award may be assigned directly to a team rather than an individual rider, so team_name cannot always be derived through bib_number. Therefore, neither dependency can be inferred from the other dependencies within its respective relation, and both must be retained in the minimal cover.

All relations satisfy 1NF because each attribute contains a single value. They also satisfy 2NF: relations with a single-attribute candidate key have no partial dependencies, while the non-key attributes in RESULT depend on the complete key (bib_number, stage_id).

Based on the identified functional dependencies, the LHS of every FD in the minimal cover is a candidate key and therefore a superkey. Thus, the final relations **satisfy BCNF and consequently also 3NF**. Country, team, rider, stage, and location details are stored in separate relations to reduce repetition and update anomalies. The derived attribute adj_time is calculated through a view rather than stored in RESULT to avoid redundancy. Therefore, no further decomposition is required under the stated functional dependencies.